

PAWCONNECT

A Full-Stack Platform for Animal Help, Adoption and Lost & Found

“Connect. Care. Help.”

Connecting people with animals that need help, care or a home.

PROJECT REPORT

Name: Saniya Sawal

Roll No: 25P0620018

Name: Sakshi Gaonkar

Roll No: 25P0620016

Master of Science Artificial Intelligence
2026

Project Website: <https://paw-connect-ivory.vercel.app/>

GitHub Repository: <https://github.com/sakshiSE/PawConnect>

1. Use Case Definition

1.1 Problem Statement

Animal-related information is often distributed across social-media posts, messaging groups, local communities and separate websites. A person may need different sources to report a missing animal, find an adoption listing, seek help for an injured animal or locate a veterinary service.

PawConnect provides one structured web platform for these activities. AI is useful during development as a coding copilot for understanding requirements, generating implementation ideas, debugging, code review and maintainability checks.

1.2 Scope

Included	Cats and dogs; lost and found reports; injured-animal help; adoption; search and filtering; authentication; image uploads; help responses; animal-care information; veterinary/service directory; basic security and deployment.
Excluded	Medical diagnosis or treatment; GPS tracking; automatic rescue dispatch; payments; legal adoption processing; full online veterinary consultation; complex AI decision-making.

Table 1: PawConnect scope

1.3 Main Features

Feature	Input	Output / Status
Authentication	Name, email, password	Registered and authenticated user
Lost & Found	Animal type, details, location, image	Searchable animal report
Animal Help	Injured-animal details	Help request and “I Can Help” response
Adoption	Animal listing details and image	Adoption listing
Dashboard	Logged-in user session	User’s own activity and records
Image Upload	Image file	Validated image URL stored through Cloudinary
Places & Services	Search/category	Relevant service information
Care Library	Category selection	Animal-care and food guidance
Authorization	User session + record ID	Allowed or blocked protected action

Table 2: Main PawConnect features

1.4 Non-Functional Requirements

The main non-functional requirements were usability, maintainability, security and deployment reliability. The interface uses a responsive, animal-friendly visual design with reusable cards and navigation components. Passwords are hashed, sessions are handled server-side, protected records use ownership checks, and service credentials are stored using environment variables.

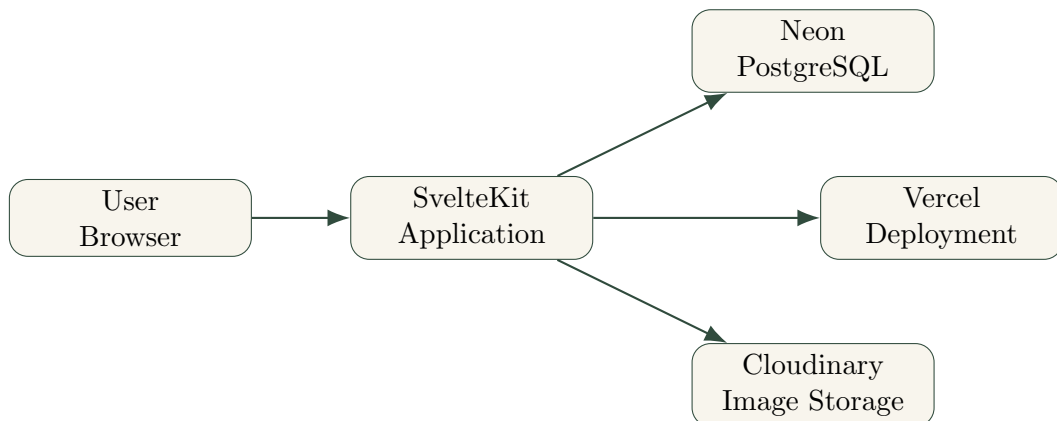


Figure 1: High-level PawConnect architecture

2. Application Functionality and Build Quality

2.1 Working Functionality

The final application provides the following working user flow:

1. A visitor opens the PawConnect landing page.
2. The user can register or log in.
3. Authenticated users can create and view animal reports.
4. Lost/found and animal-help records are stored in PostgreSQL.
5. Users can browse adoption listings and supporting information.
6. Images are validated before being uploaded to Cloudinary.
7. The dashboard provides access to the user's activity.
8. Protected modifications use authentication and ownership checks.
9. The production application is deployed through Vercel.

2.2 Application Screenshots

The following screenshots provide evidence of the implemented interface and main functionality.

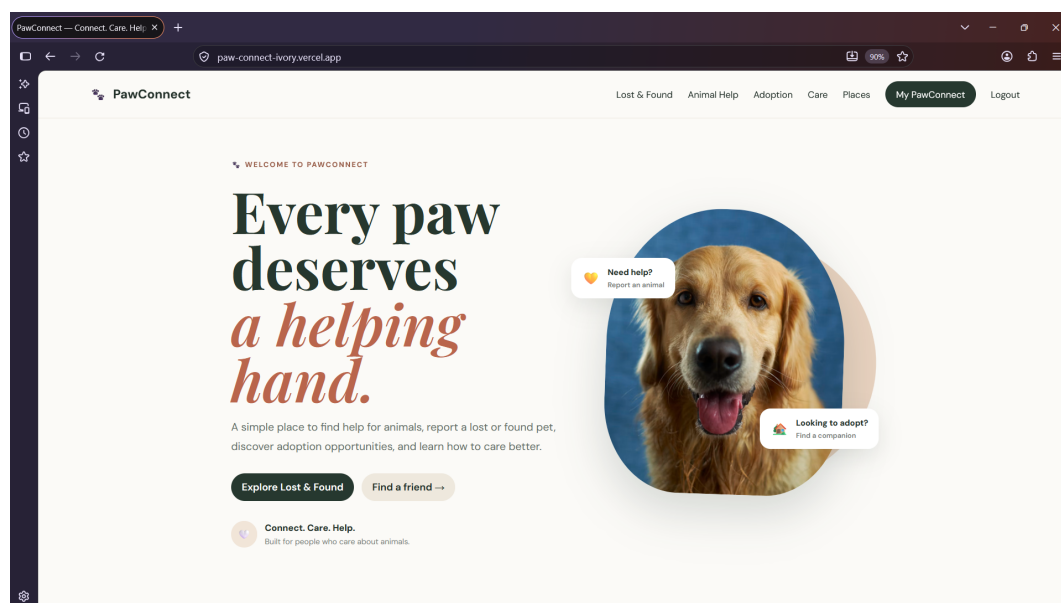


Figure 2: PawConnect landing page

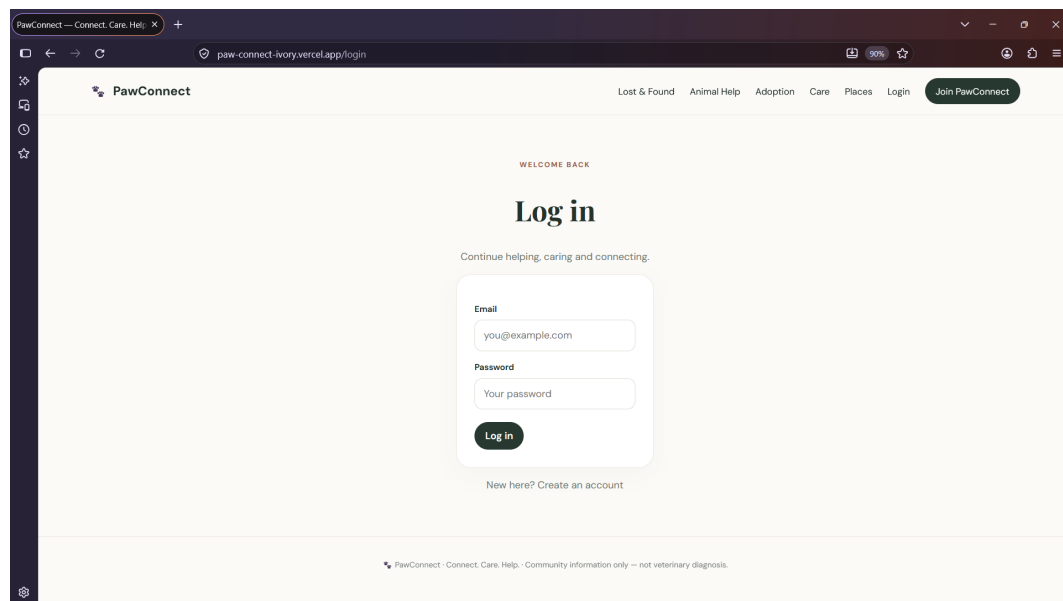


Figure 3: User login page

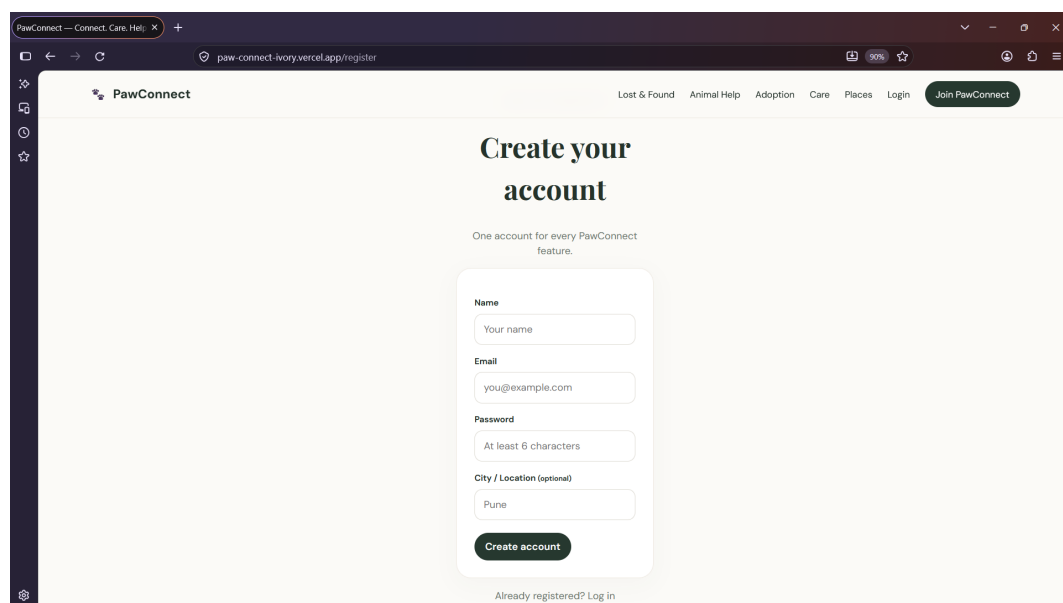


Figure 4: User registration page

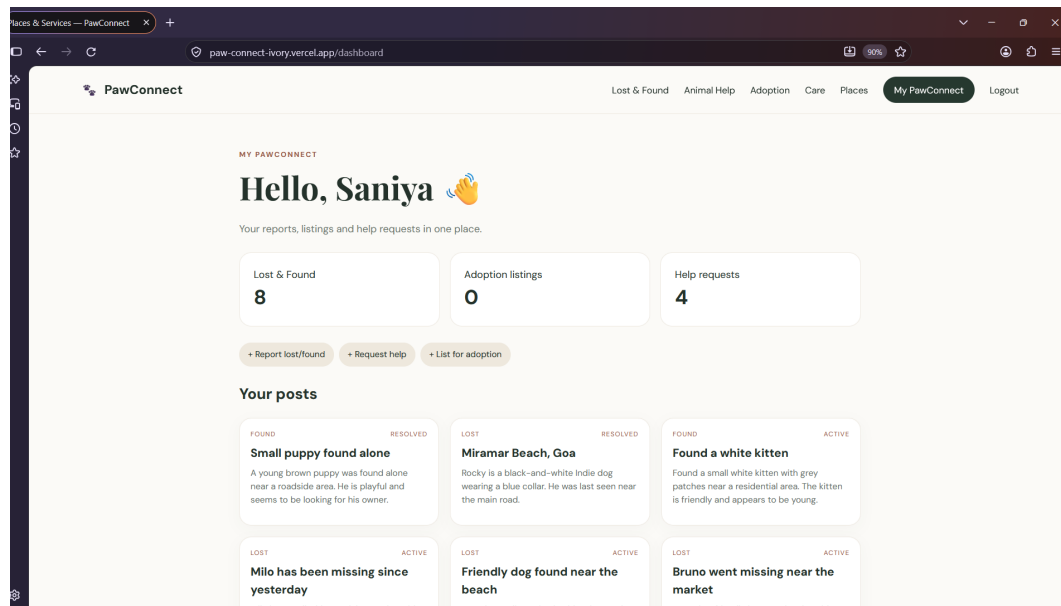


Figure 5: User dashboard

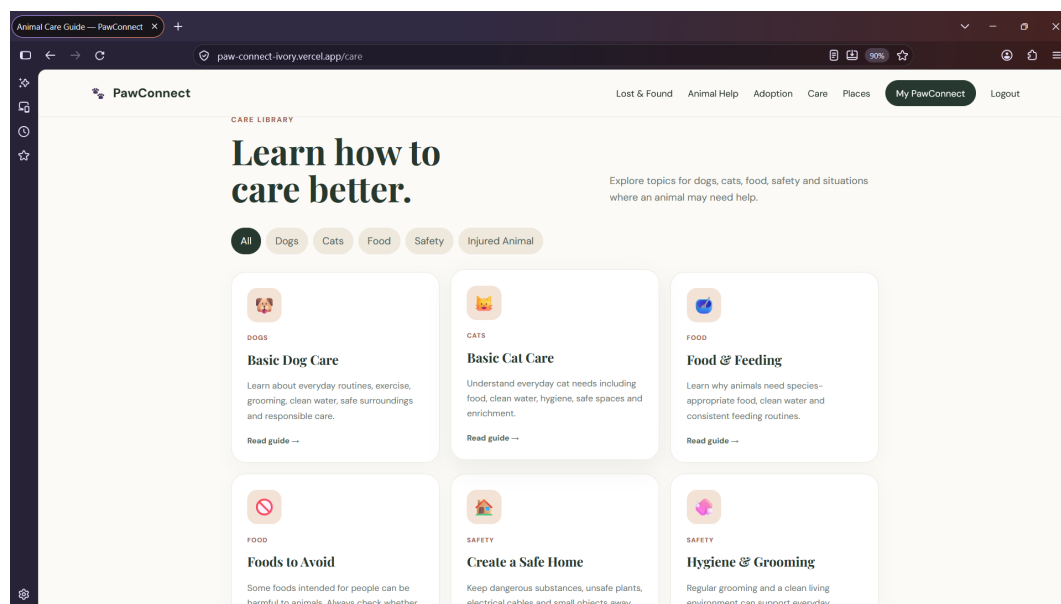


Figure 6: Animal care information

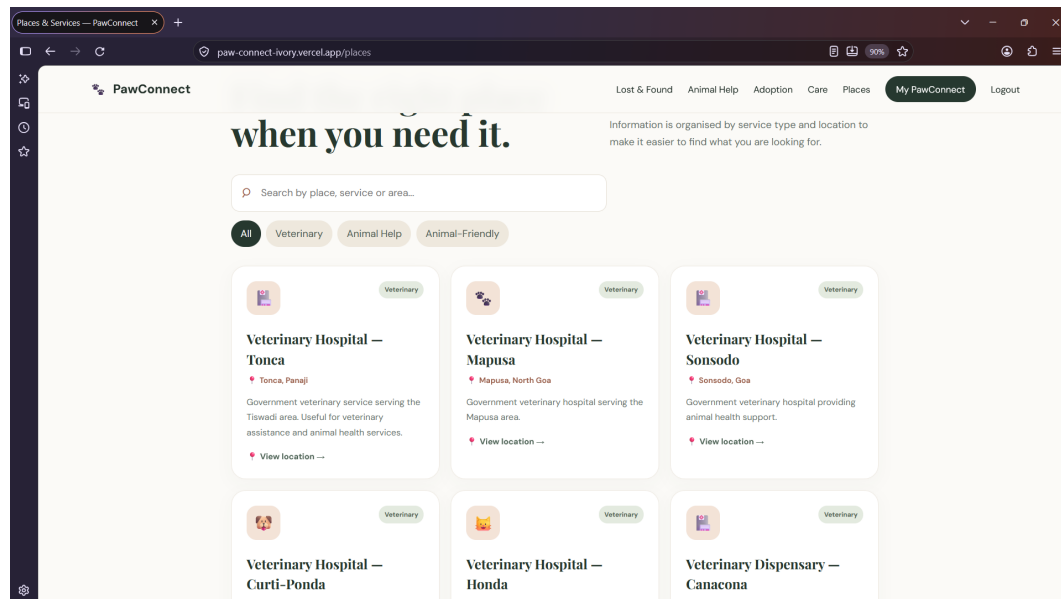


Figure 7: Places and veterinary/service information

2.3 Code Structure

The application follows a modular SvelteKit structure. UI components are separated from server-side responsibilities and database scripts.

```
pawconnect/
  database/
    schema.sql
    seed.sql
  src/
    lib/
      components/
        Card.svelte
        Nav.svelte
      server/
        auth.js
        db.js
        images.js
        animal-posts.js
    routes/
      +page.svelte
      login/
      register/
      lost-found/
      adoption/
      help/
      care/
      places/
      dashboard/
```

```
app.css
hooks.server.js
.env.example
package.json
svelte.config.js
vite.config.js
```

This structure keeps common UI, authentication, database access and image handling separate instead of placing all logic in individual pages.

2.4 Refactoring and Maintainability

Two important maintainability improvements in the final structure are:

Area	Approach	Result
Reusable UI	Common navigation and card UI are separated into <code>Nav.svelte</code> and <code>Card.svelte</code> .	Reduces repeated markup and keeps page design consistent.
Server responsibilities	Authentication, database access and image handling are kept in separate server modules.	Makes security-sensitive and infrastructure-related logic easier to locate and maintain.

Table 3: Maintainability and refactoring examples

2.5 Testing and Validation

The application was checked using endpoint testing, manual UI testing, production build validation and code review.

Thunder Client was used to verify important backend endpoints. The following screenshots show login and report endpoint testing.

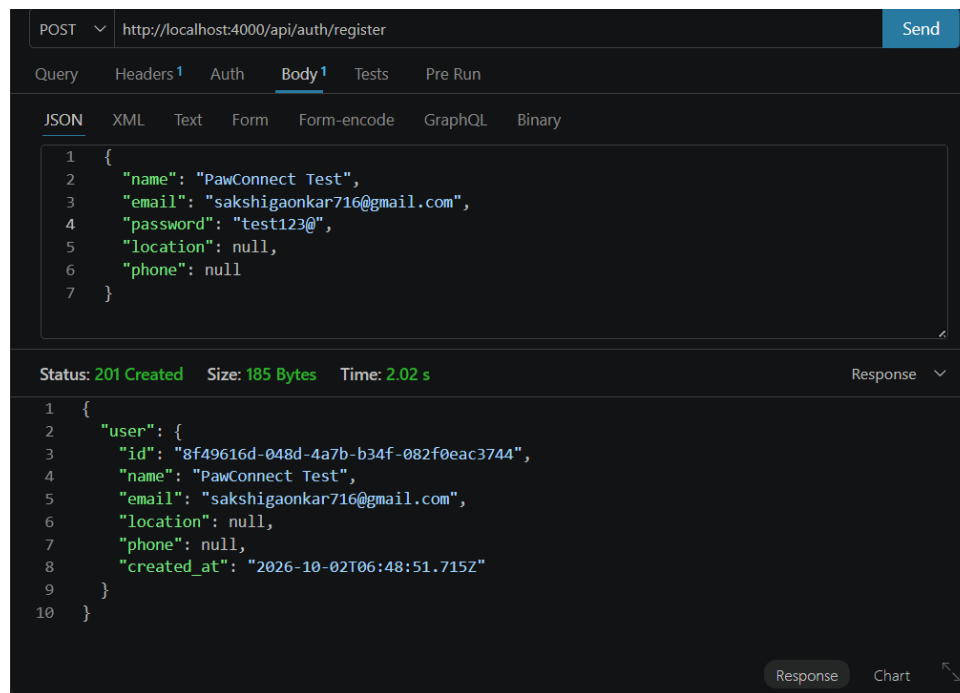


Figure 8: Registration endpoint tested using Thunder Client

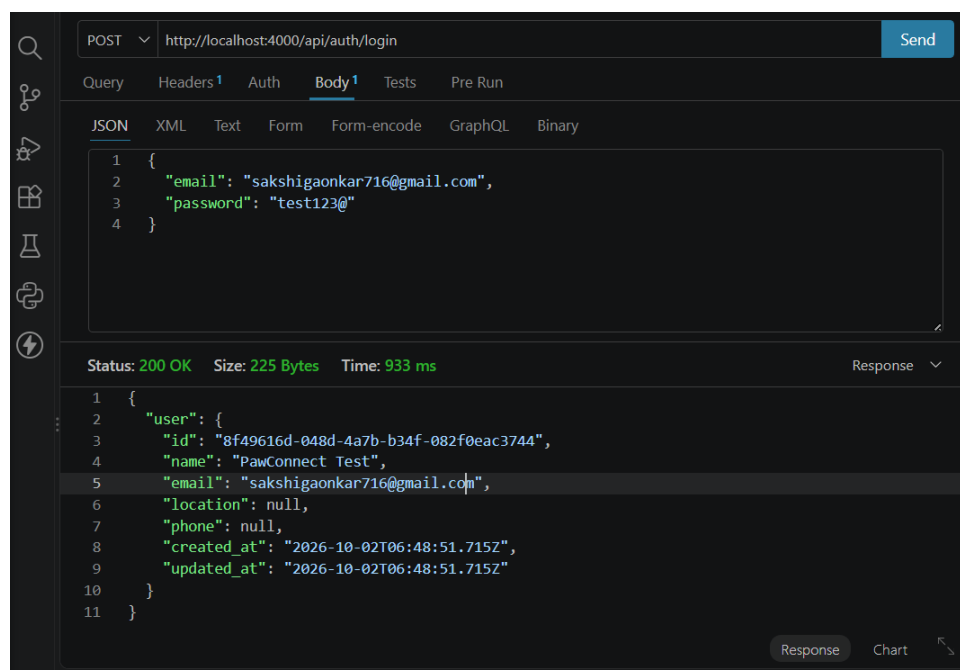


Figure 9: Login endpoint tested using Thunder Client

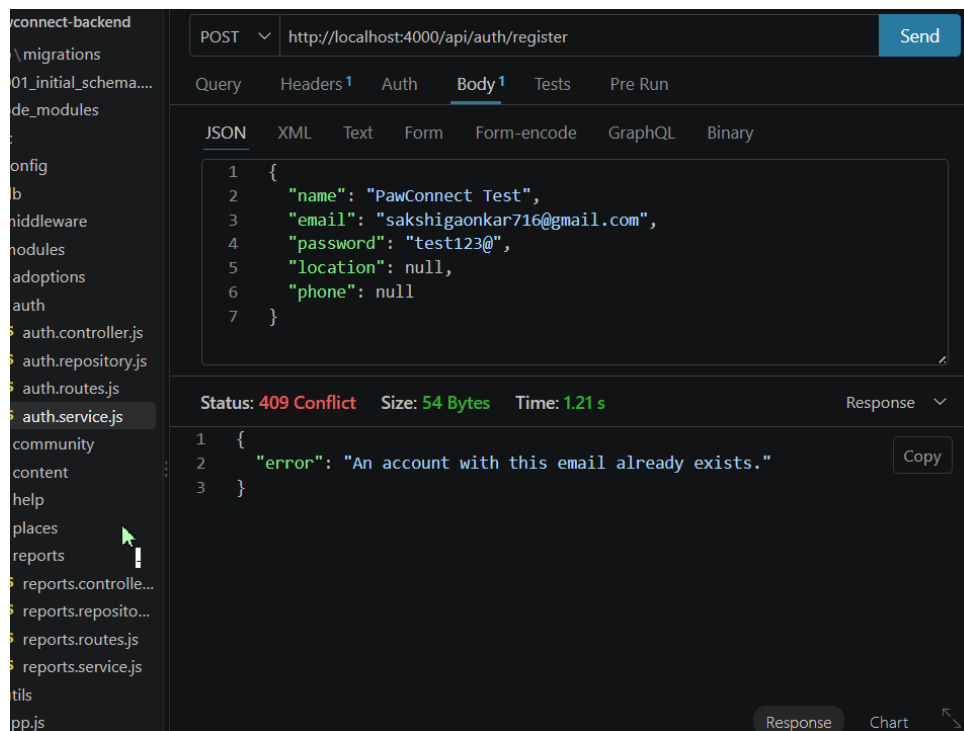


Figure 10: same email input tested using Thunder Client

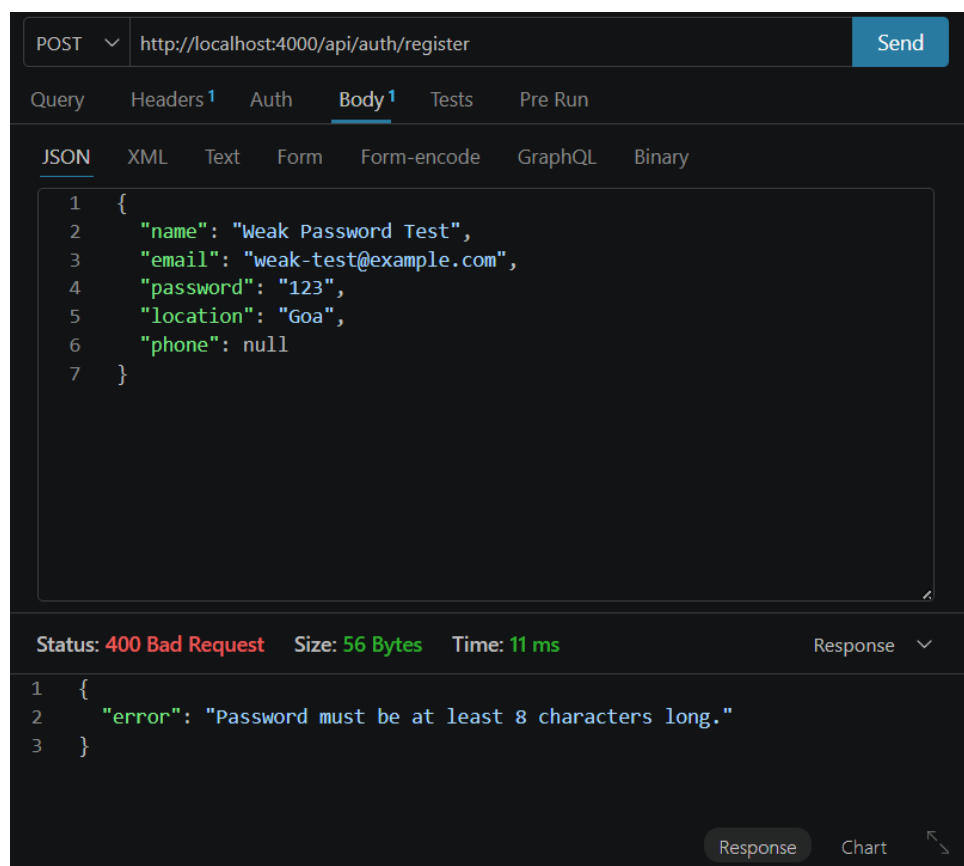


Figure 11: password validation tested using Thunder Client

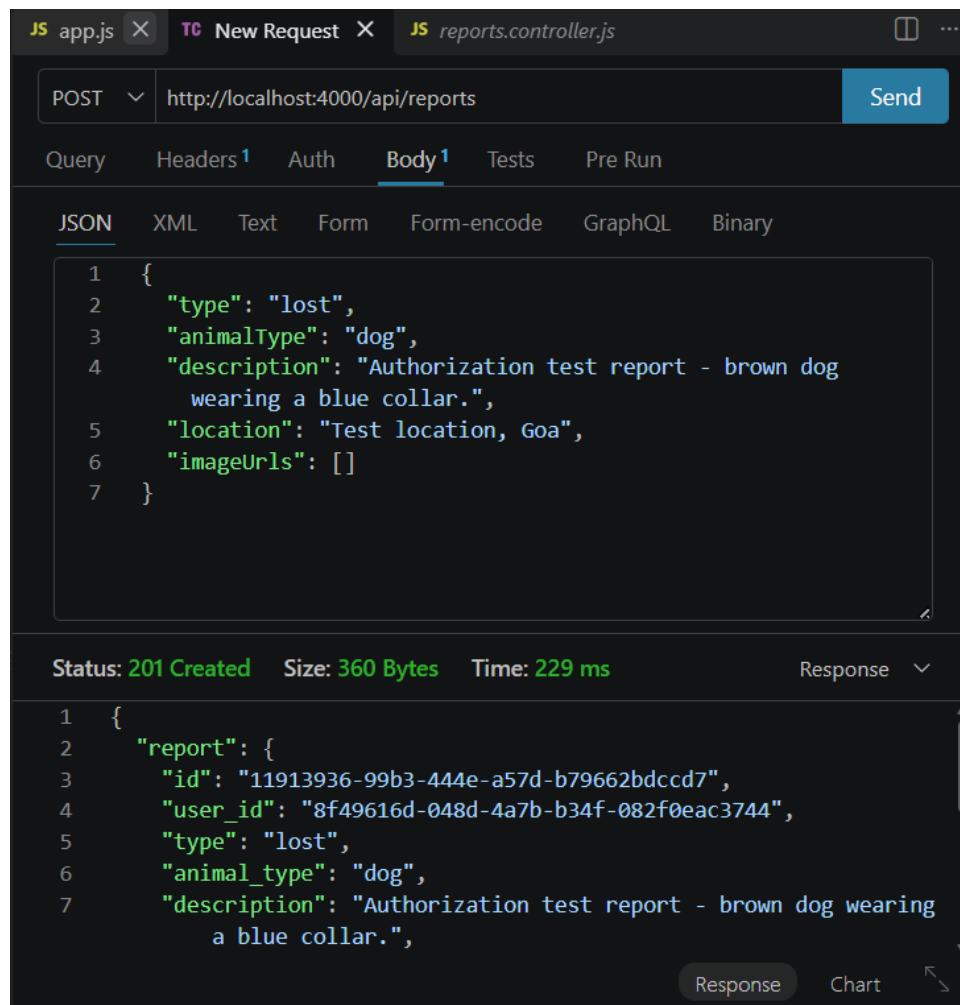


Figure 12: Report endpoint tested using Thunder Client

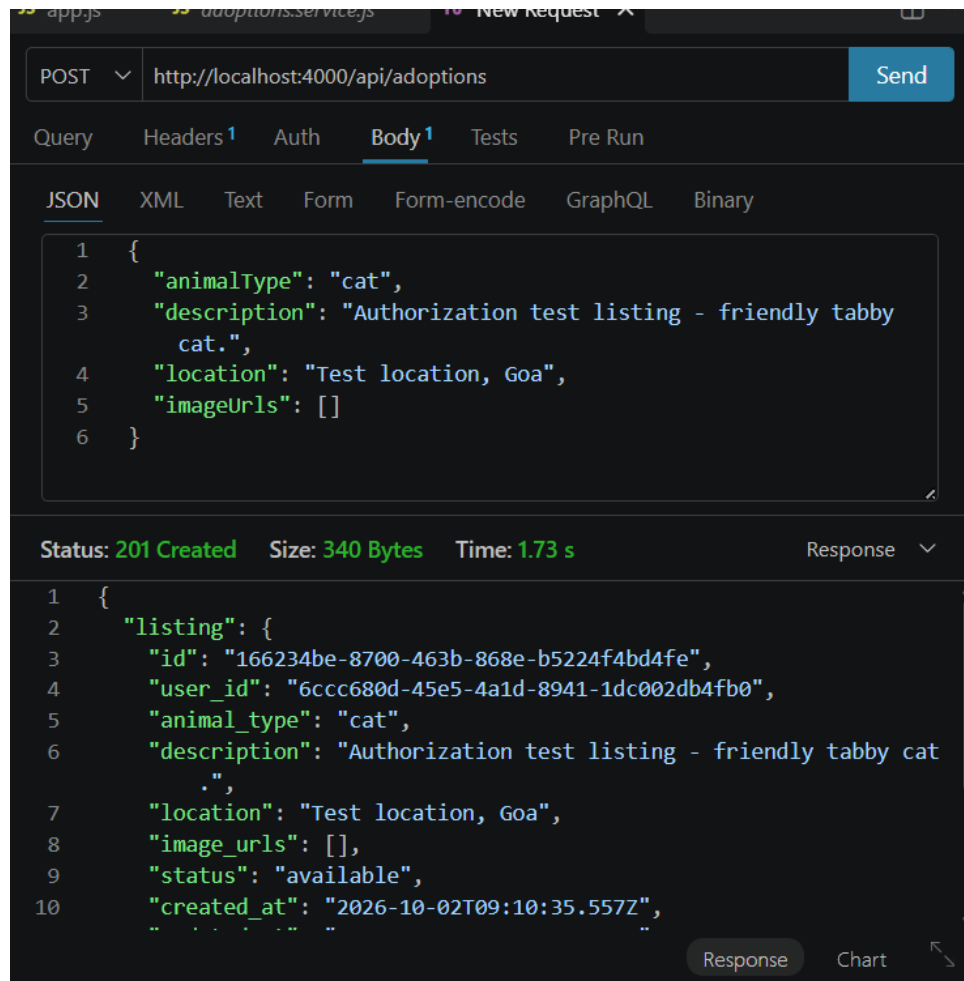


Figure 13: Adoption endpoint tested using Thunder Client

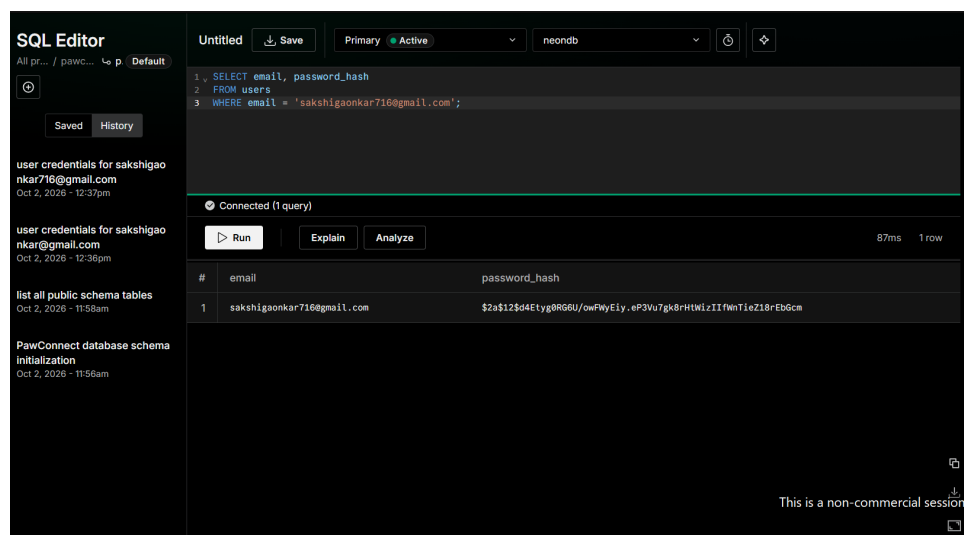


Figure 14: database checked by running query on neon sql editor

The following areas were checked:

- Registration and login.

- Invalid-input handling.
- Lost/found and help record creation.
- Search and filtering.
- Dashboard behaviour.
- Image validation and upload.
- Authentication and ownership authorization.
- Responsive user interface.
- Production build and Vercel deployment.

2.6 TDD Evidence

Formal test-first development was not consistently followed throughout the project, so the report does not claim full TDD compliance. Testing was mainly performed through endpoint testing, manual functional testing, build validation and code review after implementation.

This limitation is recorded explicitly rather than presenting a test-first history that was not maintained.

3. Prompting Strategies for Build Quality

3.1 Prompting Strategies

The AI copilot was used as a development assistant rather than as an independent decision-maker. Prompts were used for:

- Understanding the project requirements.
- Planning the technology stack and project structure.
- Generating initial implementation ideas.
- Connecting the application to Neon PostgreSQL.
- Implementing image upload and validation.
- Debugging build and deployment problems.
- Reviewing authentication, authorization and validation.
- Reviewing maintainability and repeated code.

3.2 Prompt-to-Practice Mapping

Prompt Purpose	Development Practice	Use in PawConnect
Understand requirements	Requirement analysis	Identified core MVP and scope boundaries.
Plan architecture	Modular design	Separated routes, components and server modules.
Generate implementation	Code generation	Used as a starting point for application functionality.
Debug build errors	Debugging	Investigated the Windows symlink/EPERM build issue.
Review core flows	Code review	Checked registration, login, reports and dashboard.
Review maintainability	Refactoring	Looked for repeated code and unclear responsibilities.

Table 4: Prompt-to-practice mapping

3.3 What Worked and What Required Verification

AI assistance was useful for quickly producing implementation suggestions, explaining errors and reviewing application flows. However, generated suggestions were not accepted blindly. The developer checked the code against the actual project requirements and verified important functionality through the running application and Thunder Client.

A practical example was the Windows build error involving a symbolic-link operation. Instead of changing the application functionality, the problem was treated as a local build-environment issue and the build was successfully completed using an appropriate Windows permission context.

3.4 AI Copilot Evidence

The project also used OpenAI Codex for repository-level review and maintainability assistance.

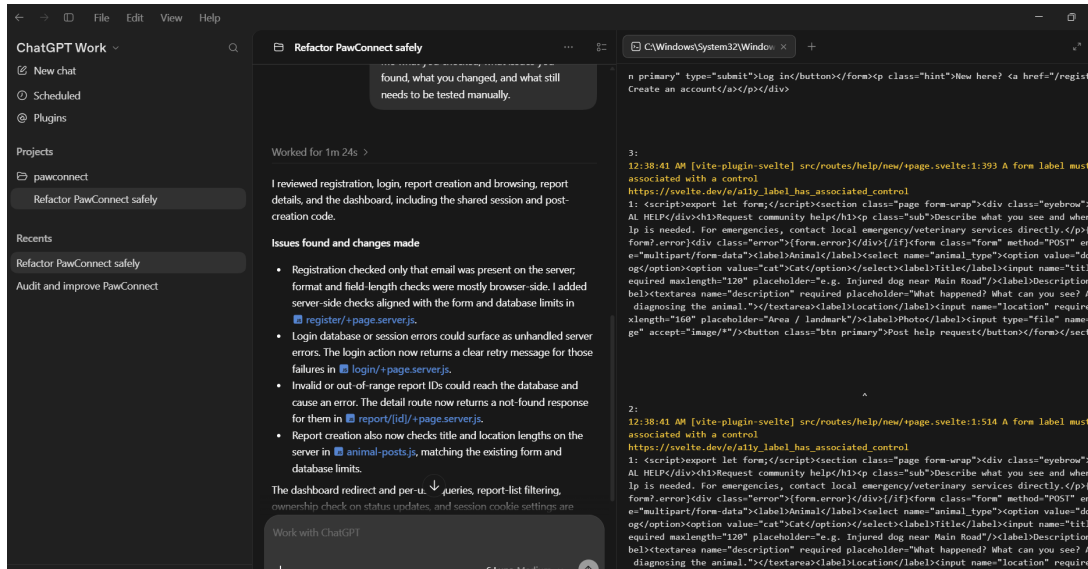


Figure 15: AI coding copilot used for PawConnect review

The copilot was used to inspect the project and provide suggestions. The developer remained responsible for deciding which suggestions were suitable for the final application.

4. Audit and Evaluation Pipeline

4.1 Evaluation Pipeline

The application was evaluated through the following sequence:

1. Requirements and scope review
- ↓
2. Code structure and security review
- ↓
3. Production build validation
- ↓
4. Thunder Client endpoint testing
- ↓
5. Manual UI and user-flow testing
- ↓
6. AI-assisted audit and review
- ↓
7. Fix or document identified issues

Figure 16: PawConnect evaluation pipeline

4.2 Evaluation Criteria

The evaluation considered:

- **Domain correctness:** features match the animal-help use case.
- **Functional quality:** core pages and workflows operate correctly.
- **Instruction following:** implementation remains within the defined scope.
- **Security:** authentication, authorization, password handling, validation and secret management.
- **Maintainability:** modular structure, reusable UI and separated server responsibilities.
- **Deployment:** successful production build and Vercel deployment.
- **Cost and latency:** no runtime LLM dependency is required by the application, reducing ongoing AI inference cost.

4.3 Model Selection

The project used an AI coding copilot based on OpenAI Codex with GPT-5.6 Luna for development assistance and review.

The selection considered practical coding ability, repository understanding, multi-file reasoning, speed and ease of use. project-specific testing remained the main basis for accepting suggestions.

4.4 Broken Windows Found

The following issues were identified during development and review:

Issue	Status	Action
Windows symbolic-link/EPERM build error	Fixed	The build environment was corrected and the production build completed successfully.
Environment configuration for database and Cloudinary	Fixed	Required credentials were moved to environment variables.
Authentication and ownership protection	Applied	Protected operations use server-side authentication and ownership checks.
External directory information	Accepted	Directory details require periodic verification because they depend on external information.

Table 5: Audit findings and actions

4.5 Fixes and Limitations

Core implementation issues identified during development were fixed where required for application functionality. Some limitations remain part of the defined project scope, including the absence of GPS tracking, automatic rescue dispatch, real-time chat and veterinary diagnosis.

5. Overall Build Process

5.1 Build Process

The project was developed incrementally:

1. **Requirement understanding:** analysed the project document and identified the core MVP.
2. **Architecture planning:** selected SvelteKit, Neon PostgreSQL, Cloudinary and Vercel.
3. **Frontend development:** created the landing page, navigation, authentication pages and feature pages.
4. **Backend development:** connected PostgreSQL and implemented authentication, sessions, records and ownership checks.
5. **Image handling:** added image validation and Cloudinary integration.
6. **Testing and debugging:** used Thunder Client, manual testing and build validation.
7. **AI-assisted review:** used Codex to review important flows and maintainability.
8. **Deployment:** configured environment variables and deployed the production application to Vercel.

5.2 Technology Stack

Layer	Technology	Purpose
Frontend / Full Stack	SvelteKit	UI, routing and server-side application logic
Language	JavaScript / Svelte	Application implementation
Database	Neon PostgreSQL	Persistent relational data
Authentication	Server-side sessions + bcrypt	Login and password security
Image Storage	Cloudinary	Animal image storage
Deployment	Vercel	Production hosting
Development	VS Code, Node.js, npm	Application development
Version Control	Git and GitHub	Source control and project history
API Testing	Thunder Client	Endpoint validation
AI Copilot	OpenAI Codex / GPT-5.6 Luna	Coding assistance and review

Table 6: PawConnect technology stack

5.3 AI Copilot Used

OpenAI Codex was used as a coding assistant for repository understanding, implementation support, debugging, code review and maintainability analysis. The developer reviewed the generated suggestions before accepting them.

The final application does not use an LLM as a runtime service.

5.4 Models Used

Model / Tool	Role	Use
OpenAI Codex	AI coding copilot	Project understanding, coding assistance, debugging and review
GPT-5.6 Luna	Coding/reasoning model	Repository-level coding and review assistance through the copilot

Table 7: AI models/tools used during development

5.5 Prompt Log

The following are the important prompts used during the development process.

Prompt 1 – Understanding the project

I have this project document. Go through the doc and understand the whole idea and requirements properly. I need to build this project as a full stack website. First understand what the project is asking, what features are important, what should be core MVP and what can be simple. Also keep in mind I am a CS student so don't make it unnecessarily complicated.

Prompt 2 – Technology and structure

Based on this project idea can you give me the proper technology stack and project structure. I want to use SvelteKit and keep the code simple and understandable. I need frontend backend database image upload authentication and deployment. Please avoid unnecessary things and tell me what files and folders I actually need.

Prompt 3 – Initial project implementation

this is what we decided can you give me proper code files which i can paste in vs code and run. keep the code simple but i want the website to look like pinterest or a nice modern website. please keep it simple, avoid duplicates, modular and easy to understand. use SvelteKit framework. I need the main pages like home, login, register, lost and found, animal help, adoption, care, places and dashboard and connect them with the backend.

Prompt 4 – Database and backend

now help me connect the project with Neon PostgreSQL and make the actual backend work. I need registration login sessions and the animal reports, adoption listings and help requests to be stored in the database. Please keep one PawConnect user type and make sure users can only change their own reports. Also explain the setup step by step because I need to run this in VS Code.

Prompt 5 – Image upload and deployment

I also need image upload in the project. Use Cloudinary for the images and keep the credentials in env variables. Add the validation also so only images and reasonable file sizes are allowed. After that help me deploy the SvelteKit project to Vercel without changing the existing functionality.

Prompt 6 – Build error debugging

i am getting this symlink EPERM error while doing npm run build in my PawConnect SvelteKit project. the project is working normally but the build is failing. can you check what this error means and tell me the easiest way to fix it without changing my project code or functionality

Prompt 7 – Code review

I'm working on my PawConnect project and want to improve the code without changing the existing functionality or design. Please look through the current SvelteKit project and check for repeated code, validation problems, authentication or authorization issues and anything that could make the project harder to maintain. Make only safe suggestions or improvements and explain what you checked.

5.6 Overall Outcome

PawConnect demonstrates a complete full-stack implementation for a practical community problem. The final application combines a responsive SvelteKit interface with PostgreSQL persistence, authentication, authorization, image storage, search/filtering and cloud deployment.

The project brings lost-and-found reporting, injured-animal help, adoption and animal-care information into one consistent platform while applying basic software-engineering, security and maintainability practices.

References

- SvelteKit Documentation: <https://svelte.dev/docs/kit>
- Svelte Documentation: <https://svelte.dev/docs/svelte>
- Neon PostgreSQL Documentation: <https://neon.com/docs>
- Cloudinary Documentation: <https://cloudinary.com/documentation>
- Vercel Documentation: <https://vercel.com/docs>
- OpenAI Documentation: <https://platform.openai.com/docs>